
Prueba para candidato a R&D Science.

Alejandro Otero Vázquez

Fecha de realización: 8 de abril de 2024

1. Consideraciones iniciales.

El programa en su totalidad ha sido escrito en C#. No se han utilizado librerías avanzadas que implementen funcionalidades de alto nivel. Se han utilizado las siguientes librerías: `stdio.h`, `stdlib.h`, `ctype.h` y `math.h`.

El encabezado `<ctype.h>` en C proporciona funciones para la clasificación de caracteres. Las funciones que se han utilizado en este programa que se encuentran en `<ctype.h>` son:

- **`isdigit(int c)`**: Devuelve un valor distinto de cero si `c` es un dígito.
- **`isspace(int c)`**: Devuelve un valor distinto de cero si `c` es un espacio en blanco.

El encabezado `<math.h>` en C proporciona una variedad de funciones y constantes matemáticas. Las funciones que se han utilizado en este programa que se encuentran en `<math.h>` son:

- **`tan(float angle)`**: Devuelve el valor de la tangente de `angle`.
- **`'M_PI'`**: Constante con valor π .

1.1. Definición de estructura `Point3D`.

Durante todo el programa se trabajará con punteros a un tipo de estructura definida como `Point3D`. Esto facilita la implementación de los cálculos que requiere esta prueba a superficies que no necesariamente sean polinómicas.

La estructura `Point3D` contiene tres variables `(x,y,z)` tipo `float`, que corresponden a las coordenadas asociadas a un punto en el espacio cartesiano tridimensional. Además contiene un puntero `double *surfaceCoefficients` que apunta a la dirección de memoria del array donde se guardarán los datos de los coeficientes a_{ij} del polinomio $z(x, y)$. Por último contiene una variable `int dimension` que almacena la dimensión de la matriz $N \times N$ de los coeficientes.

1.2. ¿Por qué trabajar con punteros a estructuras?

En líneas generales las funciones que implementa el código trabajan con argumentos puntero a estructuras `Point3D` en lugar de trabajar con variables locales. Esto se debe a que trabajar con una estructura `Point3D` requiere cargar sobre las direcciones de memoria `double *surfaceCoefficients` los valores de los coeficientes cada vez que se quiera definir un nuevo `Point3D`. Inicializar una estructura dentro de cada una de las funciones tendría una carga computacional muy alta. En su lugar, al trabajar con punteros, las funciones podrán tanto acceder como modificar la información que se encuentre en las direcciones de memoria de la estructura `Point3D` sin necesidad de crear una nueva.

1.3. Definición de puntos en R^3 .

Antes de poder empezar a utilizar la estructura Point3D esta debe ser inicializada como ya se ha mencionado. La función Point3D *definePoint3D() retorna un puntero que apunta a la dirección de memoria de una estructura tipo Point3D con unos valores de coordenadas iniciales $(x, y, z) = (0,0,0)$. Esta función ejecuta dentro de sí la función void chargeValue para que la estructura contenga también la información asociada a los coeficientes de la matriz y la dimensión. De este modo cuando se utilice como argumento este puntero, las funciones tendrán acceso a los coeficientes del archivo y podrán modificar las coordenadas (x, y, z) del punto sin necesidad de cargar de nuevo los coeficientes, simplemente accediendo a su dirección de memoria.

Para modificar las coordenadas (x,y,z) de un punto no será necesario inicializar de nuevo la estructura. La función void modifyCoordinates(float x, float y, float z, Point3D *r) accede a la dirección de memoria donde se guardan las coordenadas de un punto y las modifica dándoles el valor de los argumentos de entrada float x, float y, float z. Como accede al puntero de la estructura esta función no retorna nada.

2. Ejercicio 1.

Para el primer ejercicio se pide evaluar una superficie polinómica $z(x,y)$ cuya altura viene dada por:

$$z(x, y) = \sum_{i,j=0}^N a_{ij} x^i y^j \quad (1)$$

La superficie tiene un dominio (x,y) definido entre $((-30, -30), (30, 30))$. Los coeficientes están almacenados en un archivo externo que contiene una matriz cuadrada $N \times N$.

2.1. Obtención de los coeficientes.

La función chargeValue(Point3D *r) realiza la carga de datos desde un archivo de texto llamado sup.txt. Esta función toma como argumento un puntero a una estructura de tipo Point3D. La estructura Point3D tiene varios campos, esta función modifica la información que se almacena en la dirección de memoria en dos de ellos: dimension, que almacena la dimensión de la matriz de coeficientes, y surfaceCoefficients, un puntero a un array que contiene los coeficientes de la superficie.

El comportamiento de la función se puede dividir en los siguientes pasos:

1. Se declara una variable de tipo FILE para manipular el archivo y un buffer de caracteres para almacenar temporalmente los datos leídos de la primera línea del archivo.
2. Se inicializa el campo dimension de la estructura Point3D a 0.
3. Se abre el archivo sup.txt en modo lectura ("r"). Si el archivo no puede ser abierto, se muestra un mensaje de error y se termina la ejecución del programa.
4. Se lee la primera línea del archivo y se analiza para determinar la dimensión de la superficie. La dimensión se calcula contando el número de números que aparecen en la línea.
5. Se reserva memoria dinámicamente para el array surfaceCoefficients, con un tamaño de dimension^2 , para almacenar los coeficientes de la superficie. Utilizar memoria dinámica hace que este código pueda ser utilizado para cargar matrices de dimensión arbitraria.
6. Se leen los coeficientes de la superficie desde el archivo y se almacenan en el array surfaceCoefficients.
7. Se cierra el archivo.

2.2. Evaluación de la superficie polinómica

La función `void evaluate(Point3D *r)` se encarga de evaluar una función de superficie en un punto (x, y) , donde x e y son las coordenadas de un punto en el plano y z es el valor de la función en ese punto. Esta función toma como argumento un puntero a una estructura de tipo `Point3D`.

El comportamiento de la función se puede resumir de la siguiente manera:

1. Se inicializa la variable `r->z` a 0.0 para almacenar el valor de la función en el punto (x, y) .
2. Se utiliza un bucle doble para iterar sobre todos los coeficientes de la función de superficie, que se almacenan en el array `r->surfaceCoefficients`.
3. Se calcula el valor de la función en el punto (x, y) sumando el producto de cada coeficiente de la superficie con la potencia correspondiente de x e y .
4. El resultado se almacena en el campo `r->z` de la estructura `Point3D`.

Este es un ejemplo de la facilidad que brinda trabajar con punteros a estructuras. La función no retorna ningún valor, simplemente modifica el valor guardado en la dirección de memoria de la variable z dentro de la estructura `Point3D`.

2.3. Análisis de resultados.

Como el programa se ha desgranado en una serie de funciones auxiliares que implementan cada una de las funcionalidades que requiere el programa, la función `main()` presenta una estructura muy sencilla. Primeramente define un puntero a estructura `Point3D` que inicializa igualándolo a la función `definePoint3D()`. Posteriormente asigna distintas coordenadas al punto y evalúa la superficie $z(x, y)$ en esos puntos. El programa devuelve los siguientes valores:

```
r = (x, y, z)
```

```
r = (-10.000000, 5.000000, 0.775262),    z(-10.000000, 5.000000) = 0.775262
```

```
r = (5.000000, 10.000000, 0.785539),    z(5.000000, 10.000000) = 0.785539
```

```
r = (10.000000, 10.000000, 1.179258),    z(10.000000, 10.000000) = 1.179258
```

3. Ejercicio 2.

El propósito de este ejercicio será encontrar las coordenadas del punto de corte entre la superficie polinómica y un rayo cuyo origen está en $(0, 0, 20)$ y cuya dirección es $(\tan(u), \tan(v), 1)$, siendo (u, v) ángulos.

La función `Point3D findIntersection(float u, float v, Point3D *function, Point3D origin)` aborda totalmente el problema. Este punto de intersección se determina iterativamente mediante el método de la bisección.

La función toma los siguientes parámetros como entrada:

- `u` y `v`: Ángulos de dirección que definen la dirección de la línea de intersección.
- `function`: Un puntero a una estructura `Point3D` que contiene la función de superficie y sus coeficientes.
- `origin`: El punto de origen desde el cual se traza la línea de intersección.

1. La función calcula inicialmente los componentes de dirección de la línea en el espacio tridimensional usando las funciones tangente para los ángulos u y v .

2. Luego, calcula un valor inicial t basado en la distancia máxima desde el origen hasta los límites del espacio tridimensional en que está definida la superficie y en dirección de la línea de intersección.
3. Dentro de un bucle do-while, la función evalúa la función de superficie en el punto definido por t y calcula también el valor z del rayo. Luego, ajusta t y repite este proceso hasta que la diferencia absoluta entre el valor de z de la función de superficie y el valor z calculado en el punto sea menor que una cierta tolerancia h .
4. Una vez que se encuentra el punto de intersección dentro de la tolerancia especificada, los valores de x , y , y z de este punto se almacenan en una estructura Point3D, que luego se devuelve como resultado de la función.

Esta variable Point3D que contiene las coordenadas del punto de intersección no necesita ser inicializada porque no contendrá los valores de los coeficientes de la función. Simplemente unas coordenadas.

3.1. Análisis de resultados.

Estableciendo una incertidumbre de 10^{-4} se obtienen los siguientes puntos de intersección:

Punto de intersección para los ángulos (u : -20, v : 10),
 $r = (7.174927, -3.475925, 0.287048)$
 En ese punto $z(x, y) = 0.287056$

Punto de intersección para los ángulos (u : 10, v : 20),
 $r = (-3.481347, -7.186118, 0.256302)$
 En ese punto $z(x, y) = 0.256330$

Punto de intersección para los ángulos (u : -15, v : -15),
 $r = (5.250794, 5.250794, 0.403770)$
 En ese punto $z(x, y) = 0.403802$

4. Ejercicio 3.

En este último punto se pide calcular las curvaturas principales (k_1, k_2) , de la superficie en unos puntos determinados.

En cada punto p de una superficie diferenciable en un espacio euclídeo tridimensional se puede elegir un vector normal unitario. Un plano normal en p es aquel que contiene el vector normal y, por lo tanto, también contendrá una dirección única tangente a la superficie y cortará a la superficie generando una curva plana, llamada sección normal. En general, esta curva tendrá diferentes curvaturas para diferentes planos normales en p . Las curvaturas principales en p , denominadas k_1 y k_2 , son los valores máximo y mínimo de esta curvatura.

Las curvaturas principales son los autovalores del operador de forma y las direcciones principales son sus autovectores. Puede probarse, que este operador de forma, denominado segunda forma fundamental, resulta ser un tensor 2-covariante y simétrico cuyos coeficientes no son más que:

$$B = \begin{pmatrix} b_{11}(x, y) & b_{12}(x, y) \\ b_{21}(x, y) & b_{22}(x, y) \end{pmatrix}$$

donde cada coeficiente viene dado por las siguientes ecuaciones:

$$\begin{aligned} b_{11} &= \frac{\partial \mathbf{n}}{\partial u} \cdot \frac{\partial \mathbf{r}}{\partial u} \\ b_{12} &= \frac{\partial \mathbf{n}}{\partial v} \cdot \frac{\partial \mathbf{r}}{\partial u} \\ b_{21} &= \frac{\partial \mathbf{n}}{\partial v} \cdot \frac{\partial \mathbf{r}}{\partial v} \end{aligned}$$

En nuestro caso, la superficie está parametrizada por x , y y por lo que los coeficientes del tensor vendrán dados por:

$$\begin{aligned} b_{11} &= \frac{\partial^2 f}{\partial x^2} \\ b_{12} &= \frac{\partial^2 f}{\partial x \partial y} \\ b_{21} &= \frac{\partial^2 f}{\partial y \partial x} \\ b_{22} &= \frac{\partial^2 f}{\partial y^2} \end{aligned}$$

Como la superficie $z(x,y)$ es polinómica, pueden obtenerse analíticamente sus derivadas de cualquier orden.

1. La función `second_derivative_xx` calcula la segunda derivada parcial con respecto a x (es decir, $\frac{\partial^2 f}{\partial x^2}$) en un punto r_0 almacenado en la estructura `Point3D`. Para ello, itera sobre los coeficientes de la superficie almacenados en `r0->surfaceCoefficients` y calcula la segunda derivada con respecto a x en función de x e y , acumulando el resultado en la variable `dxx`.
2. La función `second_derivative_yy` hace lo mismo pero para la segunda derivada parcial con respecto a y (es decir, $\frac{\partial^2 f}{\partial y^2}$), acumulando el resultado en la variable `dyy`.
3. La función `second_derivative_xy` calcula la segunda derivada parcial cruzada (es decir, $\frac{\partial^2 f}{\partial x \partial y}$), acumulando el resultado en la variable `dxy`.

Existen muchas aproximaciones numéricas para calcular las segundas derivadas parciales de una función. Si la función no fuera polinómica sería necesario utilizarlas, pero dan resultados menos exactos. Dentro del programa se han dejado comentadas las funciones que implementan el cálculo de segundas derivadas por el método numérico, por si en algún momento el programa se utilizara con otras funciones. Estas ecuaciones se conocen como diferencias finitas de 5 puntos.

$$f''(x_i) \approx \frac{-f(x_{i-2}) + 16f(x_{i-1}) - 30f(x_i) + 16f(x_{i+1}) - f(x_{i+2}))}{12h^2}$$

Donde $f''(x_i)$ es la segunda derivada de f en el punto x_i , y h es el tamaño del paso.

4.1. Cálculo del tensor de forma.

Las ecuaciones para el cálculo de las segundas derivadas parciales dan los valores del tensor de forma, pero es necesario todavía generar ese tensor de forma. Para ello se ha definido una estructura `TensorCurvatura` que contiene 3 variables tipo `float` que almacenan cada una de las derivadas parciales. La función `TensorCurvatura findCurvature(Point3D *r0)`, dadas unas coordenadas, calcula los coeficientes de la matriz del operador y los almacena en el tensor de forma de la superficie en ese punto.

4.2. Cálculo de las curvaturas principales.

Las curvaturas principales serán los autovalores de este tensor de forma que contiene las segundas derivadas parciales.

La función `void mainCurvatures(Point3D *r0, double *curvaturasPrincipales)` calcula las curvaturas principales en un punto dado del espacio tridimensional. Esta función toma como argumentos un puntero a una estructura de tipo `Point3D`, que representa las coordenadas del punto en el espacio, y un puntero a un array de tipo `double`, donde se almacenarán las curvaturas principales calculadas.

El procedimiento llevado a cabo por la función se puede describir en los siguientes pasos:

1. Se llama a la función `findCurvature` para calcular los coeficientes del tensor de forma en el punto dado. Esta función devuelve un objeto de tipo `TensorCurvatura`, que contiene los coeficientes de la matriz.
2. Se calcula el discriminante y la traza de la matriz de curvatura utilizando los coeficientes obtenidos.
3. Se calculan las curvaturas principales a partir de los autovalores de la matriz.
4. Las curvaturas principales calculadas se almacenan en el array pasado como argumento a la función.

4.3. Análisis de resultados.

El programa calcula las curvaturas principales de la superficie para dos parejas de puntos:

`z(10.000000, 5.000000)`

Curvatura principal en el punto $(10, 5)$, $k_1 = 0.010844$

Curvatura principal en el punto $(10, 5)$, $k_2 = 0.007226$

`z(10.000000, -10.000000)`

Curvatura principal en el punto $(10, -10)$, $k_1 = 0.022960$

Curvatura principal en el punto $(10, -10)$, $k_2 = 0.011898$